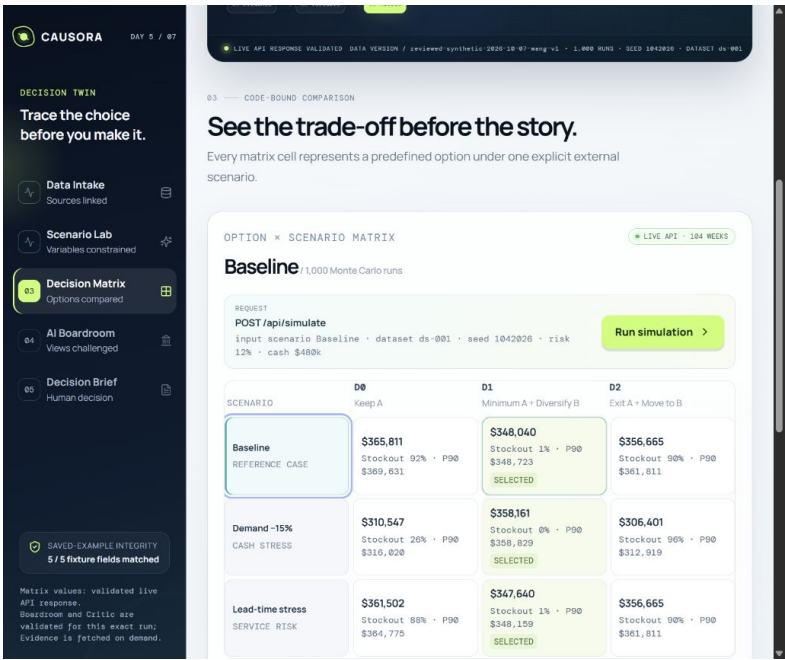


01 The decision problem

00:00-00:40 / ENGLISH NARRATION



Historical validated website capture, Day6

PRESENTER ONLY: Show the Matrix. Introduce the case, then open the production source folder Day5/Causora.

Hello, we are Ding Xiangfeng, Deng Jinzhu and Wang Hao. Causora helps procurement teams review a supplier renewal with finance and operations. A cheaper quote can hide a locked purchase obligation when demand falls. This demonstration uses reviewed synthetic data. I will show the actual production source, including simulation, real AI requests, and the conditions that enable a human decision. The website shows the result. The code explains its basis.

After saying "The code explains its basis." turn to page 02.

02 Input validation and source identity

00:40-01:10 / ENGLISH NARRATION

```
59     manifest = read_source_manifest(project_root) # pins the whole manifest, all five raw
hashes, all 104 CSV rows
60     raw = (project_root / "demo_data/causora_day1_mock.json").read_bytes()
61     if hashlib.sha256(raw).hexdigest() != FROZEN MOCK_SHA256:
62         raise ValueError("the team's v4.1 mock changed without an agreed dataVersion")
63     contract = json.loads(raw.decode("utf-8"))["contract"]
64     validate_request(request, dataset_id="ds-001", contract=contract,
allow_unfrozen_scenarios=True)
65     if parsed.datasetId != "ds-001" or policy.operatingPolicy.horizonStartDate.isoformat() !=
contract["renewalDate"]:
66         raise ValueError("named synthetic dataset and renewal-date opening inventory required")
67     if any(s["demandUnits24m"] > 1_000_000 for s in request["scenarios"]):
68         raise ValueError("trial demand exceeds safe internal arithmetic range")
69     history = manifest["historicalDemand"]["observations"]
70     leads, prices = _lead_days(manifest["supplierDelivery"]["orders"])
71     if len(history) < 52 or len(leads["A"]) < 30:
72         raise ValueError("insufficient historical weekly demand or Supplier A P0s for an
empirical bootstrap")
73     if len(leads["B"]) >= 30:
74         raise ValueError("B has enough P0s; re-review its distribution before using a user-
assumed fallback")
75     if policy.operatingPolicy.targetStockUnits <= policy.operatingPolicy.safetyStockUnits:
76         raise ValueError("target stock must exceed explicit safety stock")
77     return manifest, contract, parsed.model_dump(mode="json"), leads, prices
78
```

Pinned source excerpt / monte_carlo.py / lines 59-78

PRESENTER ONLY: Open lines 59-78. Point to model validation, read_source_manifest and the frozen source hash.

This function validates the typed request and loads the source manifest. It checks the frozen contract hash, dataset identity, and renewal date. Demand and Supplier A orders must meet sample thresholds. Supplier B lacks sufficient history, so its fallback remains an explicit assumption. These checks prevent an edited source or a different dataset from silently becoming the published calculation input.

After saying "These checks prevent an edited source or a different dataset from silently becoming the published calculation input." turn to page 03.

GitHub: backend/backend/simulation_day3/monte_carlo.py, lines 59-78

03 Shared Monte Carlo draws

01:10-01:45 / ENGLISH NARRATION

```
81     """Draw once; same external-world trials across all options and scenarios."""
82     generator = np.random.Generator(np.random.PCG64(seed))
83     n = policy.monteCarloRuns
84     hist = np.asarray([row["unitsDemanded"] for row in history], dtype=np.int64)
85     a = np.asarray(leads_a, dtype=np.int64)
86     demand_idx = generator.integers(0, len(hist), size=(n, WEEKS), dtype=np.int32)
87     lead_a_idx = generator.integers(0, len(a), size=(n, WEEKS), dtype=np.int32)
88     triangular = policy.supplierBLeadFallback
89     # B has only twelve POs; no claim that these illustrative parameters were
90     # estimated or human-approved from the small sample.
91     lead_b_days = np.clip(np.rint(generator.triangular(
92         triangular.minDays, triangular.modeDays, triangular.maxDays,
93         size=(n, WEEKS))), triangular.minDays, triangular.maxDays).astype(np.int64)
94     sampled_history = hist[demand_idx]
95     lead_a_days = a[lead_a_idx]
96     digest = hashlib.sha256(demand_idx.tobytes() + lead_a_idx.tobytes() +
97         lead_b_days.tobytes()).hexdigest()
97     return sampled_history, lead_a_days, lead_b_days, digest
```

Pinned source excerpt / monte_carlo.py / lines 81-97

PRESENTER ONLY: Stay in monte_carlo.py. Scroll to lines 81-97 and point to PCG64, the array dimensions and the digest.

PCG64 creates master draws using the recorded seed. Demand samples come from weekly history, A lead times from observed orders, and B from an explicit triangular distribution. Each trial has a path of one hundred and four weeks. Options share these external-world draws. Common random numbers reduce comparison noise caused by separate lucky samples. The digest supports repeatability checks. This still relies on historical representativeness and does not capture every future dependency.

After saying "This still relies on historical representativeness and does not capture every future dependency." turn to page 04.

GitHub: backend/backend/simulation_day3/monte_carlo.py, lines 81-97

04 Inventory state and replenishment

01:45-02:30 / ENGLISH NARRATION

```
141     delivered_a += arrival_a
142     delivered_b += arrival_b
143     available = on_hand + arrival_a + arrival_b
144     demand_week = demands[:, t]
145     filled = np.minimum(demand_week, available)
146     shortage = demand_week - filled
147     on_hand = available - filled
148     filled_total += filled
149     lost_total += shortage
150     demand_total += demand_week
151     before_order = ordered_a + ordered_b - delivered_a - delivered_b
152     position = on_hand + before_order
153     reorder = np.where(position <= point, np.maximum(0, policy.targetStockUnits - position),
0)
154     if fixed_a is not None:
155         need_a = np.maximum(0, (fixed_a * (t + 1) + WEEKS - 1) // WEEKS - ordered_a)
156         need_b = np.maximum(0, (fixed_b * (t + 1) + WEEKS - 1) // WEEKS - ordered_b)
157         qty = np.minimum(np.maximum(reorder, need_a + need_b), fixed_a + fixed_b - ordered_a
- ordered_b)
158         low = np.maximum(0, qty - (fixed_b - ordered_b))
159         high = np.minimum(qty, fixed_a - ordered_a)
160         if np.any(low > high):
161             raise AssertionError("fixed purchased-unit quota impossible")
162         a_qty = np.clip(np rint(qty * float(shares)).astype(np.int64), low, high)
163         b_qty = qty - a_qty
164     else:
165         commitment = np.maximum(0, (min_a * (t + 1) + WEEKS - 1) // WEEKS - ordered_a)
166         qty = np.maximum(reorder, commitment)
167         a_qty = qty if option["id"] == "D0" else np.zeros(runs, dtype=np.int64)
168         b_qty = qty - a_qty
169         week_purchase = a_qty * price_a + b_qty * price_b
[continued at the pinned GitHub link below]
```

Pinned source excerpt / monte_carlo.py / lines 141-172

PRESENTER ONLY: Show lines 141-172. Point to available, filled, shortage, position and fixed_a. Explain the operations rather than reading every line.

Each trial advances inventory weekly. Arrivals join opening stock. Filled demand is the smaller of demand and available inventory. Remaining demand becomes shortage. Inventory position includes stock already on order, so replenishment accounts for future deliveries. D1 respects fixed A and B purchase quotas. The other branch applies the retained-A minimum. The obligation uses the locked forecast when demand contracts. The code simulates inventory and ordering consequences rather than asking an LLM to guess cost.

After saying "The code simulates inventory and ordering consequences rather than asking an LLM to guess cost." turn to page 05.

GitHub: backend/backend/simulation_day3/monte_carlo.py, lines 141-172

GitHub: backend/backend/simulation_day3/monte_carlo.py, lines 205-207

05 Cost aggregation and risk metrics

02:30-03:00 / ENGLISH NARRATION

```
223     breakdown = {"purchase": purchase, "holding":  
_round_micro(int(np rint(np.mean(holding_micro)))),  
224     "stockoutLoss": _round_micro(int(np rint(np.mean(loss_micro)))),  
225     "renewalPremium": premium, "terminationFee": fee_usd}  
226     expected_tco = sum(breakdown.values())  
227     raw_revenue_micro = filled_total * selling_price  
228     expected_revenue = _round_micro(int(np rint(np.mean(raw_revenue_micro))))  
229     expected_gross_profit = expected_revenue - expected_tco  
230     gross_margin = expected_gross_profit / expected_revenue if expected_revenue else None  
231     stockout_count = int(np.count_nonzero(lost_total))  
232     total_demand = int(np.sum(demand_total))  
233     total_filled = int(np.sum(filled_total))  
234     cell = {"optionId": option["id"], "feasible": feasible, "expectedTco": expected_tco,  
235           "stockoutProbability": stockout_count / runs,  
236           "serviceLevel": total_filled / total_demand if total_demand else 1.0,  
237           "cashOutflowP90": p90, "breakdown": breakdown,
```

Pinned source excerpt / monte_carlo.py / lines 223-237

PRESENTER ONLY: Show lines 223-237. Mention lines 209-213 and 50-55 for the cash percentile. Point to stockoutProbability and serviceLevel separately.

The displayed total reconciles purchase, holding, shortage loss, renewal premium, and termination fee. Currency uses integer micro-dollars and explicit rounding. Cash P90 comes from trial cash spending and excludes shortage loss. Stockout probability counts trials with any lost units. Service level measures filled units divided by demand. The trace keeps component means and a sample path. A sample realised path must not be confused with the aggregate result.

After saying "A sample realised path must not be confused with the aggregate result." turn to page 06.

GitHub: [backend/backend/simulation_day3/monte_carlo.py](#), lines 223-237

GitHub: [backend/backend/simulation_day3/monte_carlo.py](#), lines 209-213

GitHub: [backend/backend/simulation_day3/monte_carlo.py](#), lines 50-55

06 Feasibility before ranking

03:00-03:35 / ENGLISH NARRATION

```
193 def select_by_scenario(simulation: dict[str, Any], request: dict[str, Any]) -> dict[str,
dict[str, Any]]:
194     """Selection uses eligible cells only; ties break by ID and violations are ordered."""
195     selections = {}
196     for scenario in request["scenarios"]:
197         scenario_id = scenario["id"]
198         cells = {row["optionId"]: row for row in simulation["matrix"][scenario_id]}
199         eligible = []
200         violations = []
201         for option_id in OPTION_IDS:
202             cell = cells[option_id]
203             failed = []
204             if not cell["feasible"]:
205                 failed.append("infeasible")
206             if cell["stockoutProbability"] > request["riskThreshold"]:
207                 failed.append("stockout_threshold")
208             if cell["cashOutflowP90"] > request["budgetCeilingUsd"]:
209                 failed.append("cash_ceiling")
210             violations.extend({"optionId": option_id, "code": code} for code in failed)
211             if not failed:
212                 eligible.append(cell)
213         if eligible:
214             winner = min(eligible, key=lambda c: (c["expectedTco"], c["optionId"]))
215             selections[scenario_id] = {"scenarioId": scenario_id, "status": "selected",
"recommendedOptionId": winner["optionId"], "constraintViolations": violations}
216         else:
217             selections[scenario_id] = {"scenarioId": scenario_id, "status": "no_feasible_option",
"recommendedOptionId": None, "constraintViolations": violations}
```

Pinned source excerpt / interfaces.py / lines 193-217

PRESENTER ONLY: Open `select_by_scenario`, lines 193-217. Point to `failed`, `eligible` and `min`.

Selection belongs to code. Each option must satisfy contract feasibility, the stockout threshold, and the cash ceiling. Only surviving options enter the cost ranking. Minimum expected TCO wins, with an identifier as the tie break. If nothing survives, the response says no feasible option. In the recorded baseline, D1 is eligible. This explains why the cheapest cell in a stressed scenario need not win. The AI cannot replace constraint screening.

After saying "The AI cannot replace constraint screening." turn to page 07.

GitHub: [backend/backend/simulation_day1/interfaces.py](#), lines 193-217

07 Evidence verification against source

03:35-04:05 / ENGLISH NARRATION

```
280         if (not isinstance(source_file, str) or not isinstance(quote, str) or not quote or
281             isinstance(page, bool) or not isinstance(page, int) or page < 1):
282             raise RegistrationError("evidence_registry_invalid", "Evidence registry is invalid.")
283         path = _resolve_source(source_root, source_file)
284         source_text = _source_text(path, page=page)
285         if quote not in source_text:
286             raise RegistrationError("evidence_source_mismatch", "Evidence quote or extracted
value no longer matches its source.")
287         if evidence_id in _EVIDENCE_CONTRACT_FIELDS:
288             field, kind = _EVIDENCE_CONTRACT_FIELDS[evidence_id]
289             if not isinstance(extracted, str) or not extracted or extracted not in quote or
extracted not in source_text or field not in contract or not _value_matches_contract(extracted,
contract[field], kind):
290                 raise RegistrationError("evidence_value_invalid", "Evidence value does not match
the simulation contract.")
291         else:
292             # EV-020 is a conditional clause, never an assertion that a legal
293             # result is unconditional. The dated contract calculation remains
294             # the source of renewalLocked.
295             if quote != _EV020_QUOTE or item.get("extractedField") != "auto_renew" or extracted
is not True:
296                 raise RegistrationError("evidence_value_invalid", "Conditional renewal evidence
is invalid.")
297             normalized = _detach(dict(item))
298             # Preserve the frozen public EvidenceRecord DTO. Source hashes and
```

Pinned source excerpt / boardroom_adapter.py / lines 280-298

PRESENTER ONLY: Open lines 280-298. Point to `_source_text`, quote matching and `_value_matches_contract`.

The verifier reads the source page again. It requires the quotation to appear there and checks the extracted value against the simulation contract. Earlier checks restrict evidence identifiers and required records. It derives success after verification instead of trusting a caller-provided `quoteMatched` flag. This connects a claim to a specific source. It verifies the reviewed case and does not guarantee interpretation of arbitrary contracts.

After saying "It verifies the reviewed case and does not guarantee interpretation of arbitrary contracts." turn to page 08.

GitHub: [backend/backend/app/boardroom_adapter.py](#), lines 280-298

GitHub: [backend/backend/app/boardroom_adapter.py](#), lines 239-267

08 Backend admission to the AI pipeline

04:05-04:35 / ENGLISH NARRATION

```
795 async def boardroom(request: Request, payload: Any = Body(...)) -> JSONResponse:
796     """Run Boardroom only for the exact latest reviewed simulation in memory."""
797     try:
798         request_id = _canonical_request_id(request)
799         except BoardroomAdapterError as exc:
800             return _failure(exc, f"req-{uuid4().hex}")
801     try:
802         if not isinstance(payload, dict):
803             raise ValueError("object required")
804         boardroom_request = BoardroomRequest.model_validate(payload)
805     except (ValidationError, ValueError, TypeError):
806         return _failure(BoardroomAdapterError("invalid_input", "Invalid Boardroom request.",
status=422,
807                                     code="validation_error"), request_id)
808     try:
809         record = repository.resolve_formal(boardroom_request)
810         request_dict = boardroom_request.model_dump(mode="json")
811         envelope, provider_mode, critic_status = await _run_formal_pipeline(request_dict, record,
request_id)
812         # A new simulation may arrive while model calls are awaited. Never label
813         # the retired record as current merely because it was current at entry.
814         repository.resolve_formal(boardroom_request)
815         return JSONResponse(
816             status_code=200,
817             content=envelope,
818             headers=_headers(request_id, current_simulation_id=record.simulation_id,
819                             provider_mode=provider_mode, critic_status=critic_status),
820         )
821     except asyncio.CancelledError:
822         raise
823     except BoardroomAdapterError as exc:
```

Pinned source excerpt / boardroom_adapter.py / lines 795-823

PRESENTER ONLY: Show lines 795-823. Point to repository.resolve_formal before and after the awaited pipeline.

The Boardroom route validates the request and resolves the exact reviewed simulation. It then awaits the formal AI pipeline. Afterwards, it resolves the simulation again. That second check matters because a newer simulation may arrive during model calls. An outdated review must not return as current. The route returns a typed envelope with simulation and provider status headers, or a structured failure for the frontend.

After saying "The route returns a typed envelope with simulation and provider status headers, or a structured failure for the frontend." turn to page 09.

GitHub: backend/backend/app/boardroom_adapter.py, lines 795-823

09 Parallel roles and bounded inputs

04:35-05:10 / ENGLISH NARRATION

```
202     tasks=[asyncio.create_task(role_task(role),name='day4-'+role) for role in ROLES]
203     try:outputs=await asyncio.gather(*tasks,return_exceptions=True)
```

Pinned source excerpt / pipeline.py / lines 202-203

PRESENTER ONLY: Show lines 202-203, then briefly show SYSTEM at line 16. Use the role names to explain assigned responsibilities.

CFO, COO, and Risk run in parallel. Each receives verified facts projected for its role. The system instruction treats document text as data and binds numeric claims to code-controlled references. Every role draft passes validation. A bounded correction can repair a returned invalid draft when the budget allows. A transport failure cannot become a valid opinion. Roles contribute perspectives, while the simulator remains responsible for numerical comparison.

After saying "Roles contribute perspectives, while the simulator remains responsible for numerical comparison." turn to page 10.

GitHub: [agents/wanghao-day3/agent_day4/pipeline.py](#), lines 202-203

GitHub: [agents/wanghao-day3/agent_day4/pipeline.py](#), lines 16-16

GitHub: [agents/wanghao-day3/agent_day4/pipeline.py](#), lines 208-217

10 Critic coverage and compound risk

05:10-05:40 / ENGLISH NARRATION

```
240     def check_critic(value):
241         if set(value.checked_roles)!=set(ROLES) or len(set(value.checked_roles))!=3:raise
PipelineError('critic_incomplete')
242         required={'renewal_condition','locked_forecast','minimum_commitment','demand_change','cas
h_pressure','holding_cost','omission','conflict'}
243         if set(value.coverage)!=required or len(value.coverage)!=len(required):raise
PipelineError('critic_coverage_incomplete')
244         scenario=next(s for s in run.simulation_request['scenarios'] if s['id']==scenario_id)
245         compound_required=run.contract['renewalLocked'] and scenario['demandShock']<0 and
mechanism['committedExcessUnits']>0
246         if compound_required and not any(i.kind=='compound_risk' and len(set(i.roles))>=2 for i
in value.issues):raise PipelineError('critic_compound_risk_omitted')
247         for issue in value.issues:
248             _check_refs(issue.evidence_ids,evidence_ids,'evidence');raw=issue.model_dump()
249             _scan_fields(raw,['headline','body'],raw['claims'],registry,[],issue.evidence_ids)
```

Pinned source excerpt / pipeline.py / lines 240-249

PRESENTER ONLY: Show lines 240-249. Point to required coverage and compound_required.

The Critic examines the roles together. The checker requires coverage of renewal, locked forecast, minimum commitment, demand, cash, holding cost, omissions, and conflicts. Falling demand with excess locked commitment requires a compound risk involving multiple functions. Issue references and prose pass further checks. Gemini is the preferred Critic, with a labelled fallback. Different model families diversify the review process but do not prove independent judgement or a measured detection rate.

After saying "Different model families diversify the review process but do not prove independent judgement or a measured detection rate." turn to page 11.

GitHub: agents/wanghao-day3/agent_day4/pipeline.py, lines 240-249

GitHub: agents/wanghao-day3/agent_day4/pipeline.py, lines 257-276

11 Numeric and business guardrails

05:40-06:20 / ENGLISH NARRATION

```
305     def validate_synthesis_numbers(draft):
306         value=draft.model_dump()
307         _scan_fields(value,['recommendation','rationale'],[c for c in value['claims'] if
c['field']!='challenges'],registry,draft.metric_refs,sorted(option_ids))
308         for challenge in draft.challenges:
309             _scan_fields({'body':challenge},['body'],[],registry,[],sorted(option_ids))
310     try:
311         validate_synthesis_numbers(synthesis)
312     except PipelineError as rejected:
313         if rejected.reason!='numeric_guardrail_rejected' or execution.calls>=config.max_calls:
314             raise
315         correction=copy.deepcopy(synthesis_payload)
316         correction['rejectedDraft']=synthesis.model_dump()
317         correction['formattingErrors']=rejected.details.get('rejectedClaims',[])
318         correction['correctionTask']='Rewrite the rejected draft in qualitative English. Remove
every quantity, numeric identifier, metric placeholder and numerical word, including one, first,
second and one-time. Use fixed instead of one-time. Preserve the verified selection, action and
factual meaning. Return claims empty. This is the only correction attempt; an invalid correction
will be rejected.'
319         synthesis=await execution.call(provider,'Synthesizer',correction,SynthesisDraft,retry=False,
se,reserve=return_reserve)
320         validate_synthesis_numbers(synthesis)
321         if synthesis.option_id not in synthesis_payload['eligibleOptionIds']:raise
PipelineError('invalid_synthesizer_option')
322         _check_refs(synthesis.metric_refs,TOKENS,'metric');raw=synthesis.model_dump()
323         _scan_fields(raw,['recommendation','rationale'],[c for c in raw['claims'] if
c['field']!='challenges'],registry,synthesis.metric_refs,sorted(option_ids))
324         # Brief/Critic do not have frontend interpolation. A placeholder there cannot be rendered
faithfully.
325         if '{{' in synthesis.recommendation+synthesis.rationale:raise
PipelineError('unrendered_brief_token')
[continued at the pinned GitHub link below]
```

Pinned source excerpt / pipeline.py / lines 305-328

PRESENTER ONLY: Show lines 305-328. Then point to `_scan_fields` at lines 37-48 and `business_check`. Keep the speech on these mechanisms.

The Synthesizer must use an existing eligible option and structured response. Numeric validation scans its recommendation and rationale against the current registry. A rejected numeric draft gets at most one correction here and passes the same checks again. Business checks separately reject semantic mistakes, such as calling B-only sourcing diversification. The response exposes a passed guardrail after relevant checks succeed. These mechanisms reject specific errors. They cannot certify every qualitative sentence.

After saying "They cannot certify every qualitative sentence." turn to page 12.

GitHub: [agents/wanghao-day3/agent_day4/pipeline.py](#), lines 305-328

GitHub: [agents/wanghao-day3/agent_day4/pipeline.py](#), lines 37-48

GitHub: [agents/wanghao-day3/agent_day4/numeric.py](#), lines 116-135

12 The real OpenRouter request

06:20-06:55 / ENGLISH NARRATION

```
824         reservation_id = await self.budget.reserve(
825             stage=stage, model=self.model, amount=reserved, input_bound=input_bound,
max_output_tokens=self._max_output_tokens(stage)
826         )
827         self.calls += 1
828         record["reservationId"] = reservation_id
829         record["status"] = "started"
830         response = await
self.session.client.with_options(timeout=effective_timeout).chat.completions.create(**parameters)
831         record['returnedModel']=redact_secrets(getattr(response, 'model', None))
832         record['providerReturned']=redact_secrets(getattr(response, 'provider', None))
833         record['requestId']=redact_secrets(getattr(response, 'id', None))
834         usage=self._usage_dict(response)
835         if usage is not None:record['tokenUsage']=redact_secrets(usage)
836         verified_cost=_safe_decimal(usage.get('cost')) if usage else None
837         if verified_cost is not None and verified_cost>=0:
838             record['usageCostUsd']=str(verified_cost);record['usageCostStatus']='VERIFIED_OPE
NROUTER_USAGE_COST'
839         if verified_cost>reserved:
840             self.session._authorized=False
841             raise ProviderFailure('openrouter_cost_exceeded_reservation')
```

Pinned source excerpt / openrouter_provider.py / lines 824-841

PRESENTER ONLY: Show lines 824-841. Point directly to chat.completions.create and returned model, usage and identifier fields. Never show environment values.

This is the real provider call. The provider reserves a conservative cost bound before dispatch. The awaited chat completions request goes to OpenRouter. The transport records returned model, call identifier, usage, and verified cost. The response still needs schema and content validation. The key comes from the backend environment and stays out of frontend code. We can show external execution, but cannot inspect a model hidden thought process.

After saying "We can show external execution, but cannot inspect a model hidden thought process." turn to page 13.

GitHub: [agents/wanghao-day3/agent_day4/openrouter_provider.py](#), lines 824-841

GitHub: [agents/wanghao-day3/agent_day4/openrouter_provider.py](#), lines 919-950

13 Persistent cost accounting

06:55-07:25 / ENGLISH NARRATION

```
70     def _locked_journal_transaction(self, action):
71         try:
72             import psycopg
73             from psycopg.types.json import Jsonb
74             with psycopg.connect(self._connection_url, connect_timeout=5) as conn:
75                 with conn.cursor() as cur:
76                     cur.execute("SET LOCAL lock_timeout = '5000ms'")
77                     cur.execute("SET LOCAL statement_timeout = '7000ms'")
78                     cur.execute("SELECT payload FROM causora_ai_budget WHERE id = %s FOR UPDATE",
(SCOPE,))
79                     row = cur.fetchone()
80                     if row is None:
81                         raise ProviderFailure("openrouter_budget_journal_unavailable")
82                     journal = row[0]
83                     if (not isinstance(journal, dict) or journal.get("kind") != JOURNAL_KIND
84                         or journal.get("authorization") not in ({"scope": SCOPE, "maxCalls":
CALL_CAP, "maxUsd": "1.00"}, BALANCE_AUTHORIZATION)
85                         or not isinstance(journal.get("entries"), list)):
86                         raise ProviderFailure("openrouter_budget_journal_invalid")
87                     for entry in journal["entries"]:
88                         self._validated_entry(entry)
89                     result, write = action(journal)
90                     if write:
91                         cur.execute("UPDATE causora_ai_budget SET payload=%s WHERE id=%s",
(Jsonb(journal), SCOPE))
92                     return result
93         except ProviderFailure:
94             raise
95         except Exception:
96             # Never propagate database URLs/passwords in transport or logs.
```

Pinned source excerpt / postgres_budget.py / lines 70-96

PRESENTER ONLY: Open lines 70-96 and show SELECT ... FOR UPDATE. Mention the reserve and mark methods without exposing database credentials.

PostgreSQL stores the public budget ledger. The transaction locks its shared row before updates, preventing concurrent requests from assuming the same available budget. Reservations precede dispatch. Verified usage settles returned cost, while uncertain remote outcomes retain conservative reservations. Current key availability bounds spend. A normal review has five stages and a six-call whole-run cap. Removing a cumulative test count does not remove this per-review bound or trigger a recharge.

After saying "Removing a cumulative test count does not remove this per-review bound or trigger a recharge." turn to page 14.

GitHub: [agents/wanghao-day3/agent_day4/postgres_budget.py](#), lines 70-96

GitHub: [agents/wanghao-day3/agent_day4/postgres_budget.py](#), lines 99-125

GitHub: [agents/wanghao-day3/agent_day4/openrouter_provider.py](#), lines 412-437

14 Frontend request and stale-result control

07:25-08:00 / ENGLISH NARRATION

```
268   async function runSimulation() {
269     if (backendHealth.status !== "healthy") { setToast("Live simulation is unavailable until GET
/health confirms the service is ready. Retry the health check or use the clearly labelled local
fallback."); return; }
270     if (runState === "golden") { setToast("The saved example is read-only. Change an assumption
to return to the editable workspace before requesting a simulation."); return; }
271     const request = currentRequest;
272     const requestKey = currentRequestKey;
273     const retainedRun = currentLiveRun;
274     const sequence = ++runSequence.current;
275     clearDownstreamState();
276     setCachedGoldenActive(false);
277     setSimulationError(null);
278     setIsSimulating(true);
279     setRunState("ready");
280     const capturedHeaders: Record<string, string> = {};
281     try {
282       const result = await postSimulate(request, async (input, init) => {
283         const response = await fetch(input, init);
284         for (const key of SIMULATION_HEADER_KEYS) {
285           const value = response.headers.get(key);
286           if (value !== null) capturedHeaders[key] = value;
287         }
288         return response;
289       });
290       if (sequence !== runSequence.current || latestRequestKey.current !== requestKey) return;
291       const response = result.response;
292       setLiveRun({ request, response, integrationMode: result.integrationMode, development:
result.development, receivedAt: new Date().toISOString() });
293       setSimulationHeaders(capturedHeaders);
294       setBoardroomState({ status: "idle" });
[continued at the pinned GitHub link below]
```

Pinned source excerpt / page.tsx / lines 268-305

PRESENTER ONLY: Show runSimulation, lines 268-305. Then show the result check at lines 332-341 for runBoardroom.

The frontend captures request identity and a sequence counter before simulation. It applies a response only when that sequence and the current request still match. Boardroom also checks the selected scenario. Editing assumptions during an outstanding request prevents its old response from overwriting the workspace. These conditions explain why input changes require fresh simulation and why navigation must not silently combine an old brief with a new matrix.

After saying "These conditions explain why input changes require fresh simulation and why navigation must not silently combine an old brief with a new matrix." turn to page 15.

GitHub: frontend/app/page.tsx, lines 268-305

GitHub: frontend/app/page.tsx, lines 332-341

15 Approval conditions and local scope

08:00-08:35 / ENGLISH NARRATION

```
128   const matchingLiveRun = liveRun && JSON.stringify(liveRun.request) === currentRequestKey &&
runState !== "golden" ? liveRun : null;
129   const currentLiveRun = cachedGoldenActive && verifiedGolden ? verifiedGolden.liveRun :
matchingLiveRun;
130   const optionCosts = useMemo(() =>
Object.fromEntries((currentLiveRun?.response.data.simulation.matrix[contextScenarioId] ??
activeData.simulation.matrix[contextScenarioId])).map((row) => [row.optionId,
money(row.expectedTco)])), [activeData, currentLiveRun, contextScenarioId]);
131   const isDraft = !currentLiveRun && (demandShift !== currentScenario.demandShock ||
riskThreshold !== DEFAULT_RISK_THRESHOLD || budgetCeilingUsd !== DEFAULT_BUDGET_CEILING_USD);
132   const currentBoardroomRun = boardroomState.status === "ready" && currentLiveRun &&
isCurrentBoardroomRun(boardroomState.run, currentLiveRun, scenarioId) ? boardroomState.run :
null;
133   const boardroomFailureIsCurrent = boardroomState.status === "error" && currentLiveRun &&
boardroomState.identity.simulationId === currentLiveRun.response.data.simulation.simulationId &&
boardroomState.identity.dataVersion === currentLiveRun.response.data.simulation.dataVersion &&
boardroomState.identity.scenarioId === scenarioId && boardroomState.identity.simulationRequestId
=== currentLiveRun.response.requestId;
134   const selectedForDecision = currentLiveRun?.response.data.selections[scenarioId];
135   const canRecordDecision = !cachedGoldenActive && !isDraft && currentLiveRun?.integrationMode
=== "reviewed-v2" && currentLiveRun.development?.executionContext.decisionReady === true &&
!!currentBoardroomRun && selectedForDecision?.status === "selected" &&
!!selectedForDecision.recommendedOptionId &&
currentBoardroomRun.response.data.numericGuardrail.passed === true &&
currentBoardroomRun.response.data.numericGuardrail.rejectedClaims.length === 0;
```

Pinned source excerpt / page.tsx / lines 128-135

PRESENTER ONLY: Show lines 128-135, then decide at lines 426-444. Point to browser-local-only and localStorage.

Approval requires a reviewed, decision-ready simulation, matching Boardroom, eligible selection, and passed numeric guardrail without rejected claims. Cached and draft states remain outside the gate. The click handler checks again. It records the option and identity with browser-local-only scope and local storage. This is a demo human review record. It executes no purchase or supplier contract. A disabled button indicates a missing prerequisite.

After saying "A disabled button indicates a missing prerequisite." turn to page 16.

GitHub: frontend/app/page.tsx, lines 128-135

GitHub: frontend/app/page.tsx, lines 426-444

16 Production startup and traffic control

08:35-09:00 / ENGLISH NARRATION

```
21     env = os.environ if environ is None else environ
22     configured = env.get("CAUSORA_AI_PROVIDER", "").strip()
23     if configured and configured.casefold() != "openrouter":
24         raise ValueError("CAUSORA_AI_PROVIDER must be openrouter in production")
25     # An inherited OPENAI_API_KEY/OPENAI_API_BASE must never select the legacy
26     # generic provider when an operator omitted this non-secret selector.
27     env["CAUSORA_AI_PROVIDER"] = "openrouter"
28     env["CAUSORA_PUBLIC_REVIEW_ADMISSION"] = "YES"
29
30
31 def production_address(environ: dict[str, str] | None = None) -> tuple[str, int]:
32     """Validate the hosting address and reject development-only environment flags."""
33     env = os.environ if environ is None else environ
34     prohibited = {
35         "CAUSORA_DEV_UNREVIEWED_MODE": "The unreviewed development simulation override is
prohibited in production.",
36         "CAUSORA_DAY4_UNREVIEWED_PROVIDER_DEVELOPMENT_ONLY": "The unreviewed provider development
override is prohibited in production.",
37     }
38     for key, message in prohibited.items():
39         if env.get(key, "").strip():
40             raise ValueError(f"{key}: {message}")
41     host = env.get("HOST", "0.0.0.0").strip()
42     if host != "0.0.0.0":
43         raise ValueError("HOST must be 0.0.0.0 for a public Render web service")
```

Pinned source excerpt / render_start.py / lines 21-43

PRESENTER ONLY: Show lines 21-30 and 35-43, then review_admission.py lines 20-44. Do not open secret settings.

The launcher forces OpenRouter and rejects development overrides before serving the API. Vercel serves the frontend and Render runs Python. Admission allows one active review in the configured worker and a short cooldown. Busy requests receive a structured response before provider dispatch. The finally block releases admission on failure. This limits overlapping reviews alongside persistent cost accounting.

After saying "This limits overlapping reviews alongside persistent cost accounting." turn to page 17.

GitHub: [deployment/render_start.py](#), lines 21-43

GitHub: [backend/backend/app/review_admission.py](#), lines 20-44

17 Failure preservation and evidence of execution

09:00-09:30 / ENGLISH NARRATION

```
332     const result = await postBoardroom(run, requestedScenarioId, fetch, controller.signal,
333     requestId);
334     if (sequence !== boardroomSequence.current || !isSameActiveRun(run, requestKey) ||
335     scenarioId !== requestedScenarioId) return;
336     setBoardroomState({ status: "ready", run: result });
337     setToast(result.providerMode === "same-family-fallback" ? "Boardroom validated with an
338     explicitly labelled same-family fallback; the simulation was not rerun." : "Matching Boardroom,
339     Critic, and Numeric Guardrail response validated.");
340   } catch (error) {
341     if (sequence !== boardroomSequence.current || controller.signal.aborted ||
342     !isSameActiveRun(run, requestKey) || scenarioId !== requestedScenarioId) return;
343     const failure = asDay4Failure(error, requestId);
344     setBoardroomState({ status: "error", identity, failure });
345     setToast(`${failure.message} The validated Matrix and Formula Trace remain available.`);
346   } finally {
347     if (boardroomAbortRef.current === controller) boardroomAbortRef.current = null;
348   }
349 }
```

Pinned source excerpt / page.tsx / lines 332-343

PRESENTER ONLY: Show lines 332-343, then switch to the recorded successful Brief screenshot. Describe historical evidence honestly.

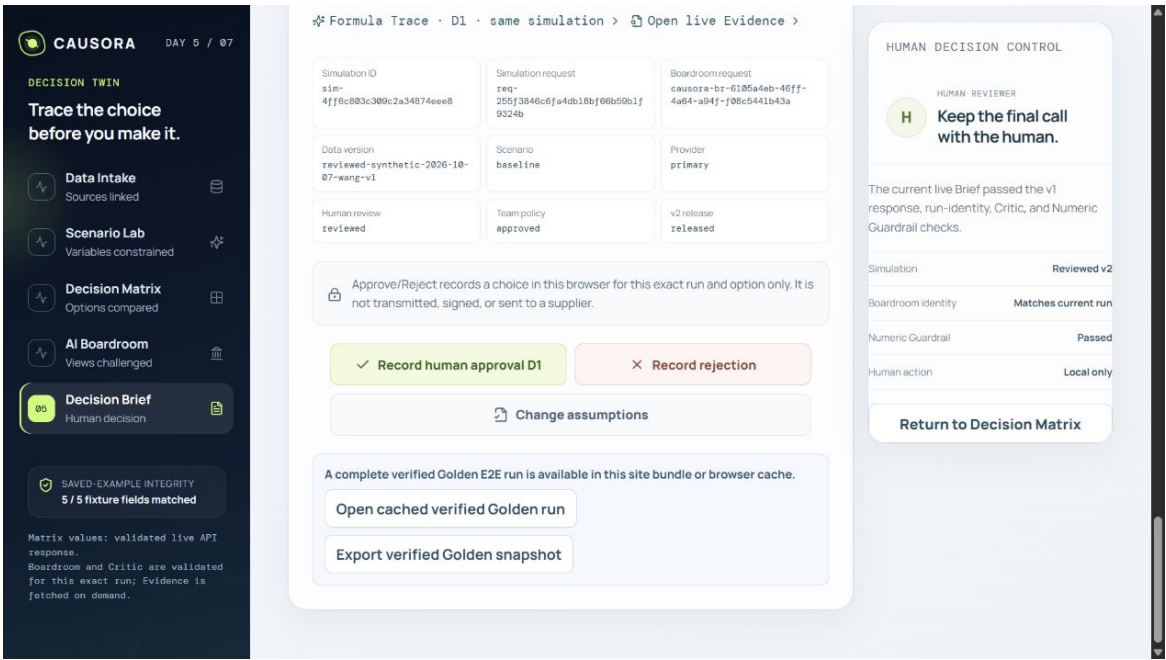
The failure branch changes Boardroom state and preserves the validated matrix and formula trace. Retrying AI does not discard simulation. Recorded verification returned all five stages for about two cents. That proves a complete real review executed. The supplied screenshot shows a historical success, not a fresh call in this video. Cached results stay read only. Public services can still fail, so retained results need clear labels.

After saying "Public services can still fail, so retained results need clear labels." turn to page 18.

GitHub: frontend/app/page.tsx, lines 332-343

18 What makes the implementation distinctive

09:30-10:00 / ENGLISH NARRATION



Historical validated website capture, Day6

PRESENTER ONLY: Return to the recorded Brief or your current validated run. End after the final sentence. Do not request another AI run just to fill time.

Causora connects procurement decisions to repeatable simulation, source evidence, constrained AI review, and a matching human record. Its distinction lies in these connected mechanisms and their inspectable code. This case uses synthetic data and limited user testing. Wider testing and independent benchmarks remain next. Today we can show real model calls and a clear boundary between computed facts, model interpretation, and human choice. Thank you.

End recording after thanking the audience.